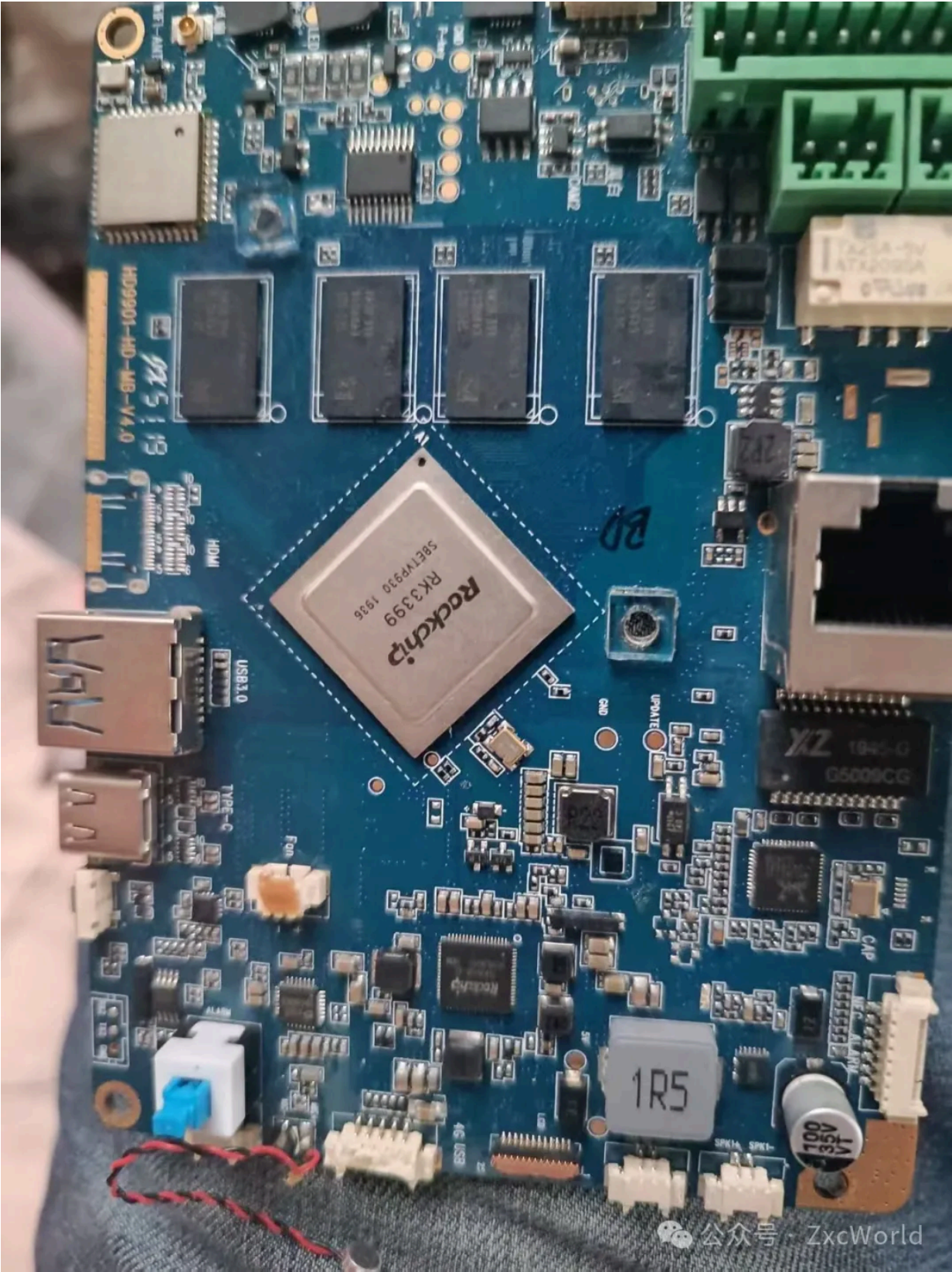


这篇写的是rk399的内核移植，我以前写过一篇A33的内核移植，我在写这篇时，已经完成我自制的v3s验证板的所有功能，然后又捣鼓了这个rk3399的移植，写的比之前的应该要好一点。这个rk3399是64位的，相比于A33或者v3s移植稍微麻烦一点，涉及到一些其他的知识。

先介绍一下这个板子，这个板子是我40元在咸鱼上买的，上面功能挺多的，可惜我既没有设备树也没有原理图，板子原来是带系统的，可惜我没有想着down下来，直接覆盖掉了。不过我在那个v3s的完整项目中，设备树、驱动都有写，这种片外外设其实大差不差，和soc没啥关系。



从这个rk3399开始，我将我所有的环境改用了docker。其实我很早就知道docker了，可是我一直认为我的需求里面并不需要它，事实证明我错得离谱。我电脑原本有3个虚拟机，后来磁盘不够，陆续干掉2个，直到捣鼓rk3399时，不知道干了什么坏事，整个虚拟机挂了，这可是糟糕透了，整个环境需要重构。所以我在这里向您强烈推荐docker，使用docker可以完美避免上面的两个弊端。有很多很多乐于分享的先驱们已经提供极丰富的教程，我也是从中学学，就不再班门弄斧。

然后以下内容，其实是我二次整理，有一部分第二次重新进行了编译，因为我添加了一些新的内容，也是在捣鼓的过程中学到的，主要是增加了initramfs和uboot的一种新的启动参数设置方式(这个新是对于我而言的)，所以可能有部分截图文件名对不上，我发现的已经改过来了，没发现的，还请见谅。

1.Docker环境

首先，我创建了一个目录docker(好吧，这个名字有点抽象，本是测试一下docker，结果一不小心，就在里面完成了)，本次所有操作在该目录完成。

进入目录，创建文件`Dockerfile`,内容如下，这就是本次构建的docker环境，其实交叉编译，很多时候是环境问题，然后那些注释掉的git库，我推荐下在本地，然后挂载到docker上去。

```
1 FROM ubuntu:22.04
2
3 # 基础设置
4 #安装默认，不要弹出对话框
5 ENV DEBIAN_FRONTEND=noninteractive
6 RUN apt-get update && apt-get install -y \
7     gcc \
8     g++ \
9     gawk \
10    libusb-1.0-0-dev \
11    dh-autoreconf \
12    libudev-dev \
13    pkg-config \
14    autoconf \
15    automake \
16    libtool \
17    debootstrap \
18    qemu-user-static \
19    binfmt-support \
20    ca-certificates \
21    gnupg \
22    bc \
23    make \
24    cmake \
25    git \
26    python3 \
27    python3-pip \
28    libssl-dev \
29    libncurses5-dev \
30    libncursesw5-dev \
31    flex \
32    bison \
33    swig \
34    device-tree-compiler \
35    libgnutls28-dev \
36    python3-serial \
37    python3-pexpect \
38    python3-pyelftools \
39    python3-setuptools \
40    python3-dev \
41    wget \
42    unzip \
43    tar \
44    locales \
45    sudo \
46    && apt-get clean
47
48 # 安装交叉编译工具链
49 #切换到/opt
50 WORKDIR /opt
51 RUN wget https://developer.arm.com/-/media/Files/downloads/gnu/14.2.rel1/binutils/aarch64-none-linux-gnu-gcc.tar.xz &&
52     tar -xvf arm-gnu-toolchain-14.2.rel1-x86_64-arm-none-eabi.tar.xz &&
53     rm arm-gnu-toolchain-14.2.rel1-x86_64-arm-none-eabi.tar.xz && \
54     wget https://developer.arm.com/-/media/Files/downloads/gnu/14.2.rel1/binutils/aarch64-none-linux-gnu-gcc.tar.xz &&
55     tar -xvf arm-gnu-toolchain-14.2.rel1-x86_64-aarch64-none-linux-gnu-gcc.tar.xz &&
56     rm arm-gnu-toolchain-14.2.rel1-x86_64-aarch64-none-linux-gnu-gcc.tar.xz &&
57
58 #该环境可能由于网络问题下载不了，可以手动下载后，挂载上去，如果可以下载，直接去掉注释
59 # RUN git clone https://github.com/TrustedFirmware-A/trusted-firmware-a.git
60 # RUN git clone https://github.com/rockchip-linux/rkbin.git
61 # RUN git clone https://github.com/rockchip-linux/rkdeveloptool.git
62 # RUN git clone https://github.com/OP-TEE/optee_os.git
```

```
63 # RUN git clone --branch v2025.04 https://github.com/u-boot/u-boot.git
64 # RUN git clone --depth 1 --branch v6.12.16 https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git
65
66 # 设置交叉编译环境变量
67 ENV PATH="/opt/arm-gnu-toolchain-14.2.rel1-x86_64-aarch64-none-linux-gnu/bin:${PATH}"
68 ENV PATH="/opt/arm-gnu-toolchain-14.2.rel1-x86_64-arm-none-eabi/bin:${PATH}"
69 ENV CROSS_COMPILE="aarch64-none-linux-gnu-"
70
71 # 创建一个编译目录
72 WORKDIR /workspace
73 COPY build-rootfs.sh .
74 RUN chmod +x ./build-rootfs.sh
75 # 默认起个 bash
76 CMD ["/bin/bash"]
77
```

再创建一个文件`build-rootfs.sh`,这个用于构建根文件debian，因为构建过程比较固定，所以我将其写成了一个脚本，不要它也是可以的，但需要将上面docker环境中关于build-rootfs.sh两行注释掉。

```
1 #!/bin/bash
2 set -e
3
4 TARGET=./rootfs-debian
5 ARCH=arm64
6 RELEASE=bookworm
7
8 mkdir -p $TARGET
9
10 debootstrap --foreign --arch=$ARCH $RELEASE $TARGET http://deb.debian.org/debian
11 # 为了在 x86 主机上 chroot 进入 arm64 文件系统，需要拷贝 QEMU
12 cp /usr/bin/qemu-aarch64-static $TARGET/usr/bin/
13 update-binfmts --enable qemu-aarch64
14 #
15 chroot $TARGET /debootstrap/debootstrap --second-stage
16
```

然后运行这个命令构建docker环境，这个命令是旧版本的，如果新版本的可以使用`docker buildx build`，新版本可能现在没有普及：

```
1 sudo docker build -t rk3399 .
```

构建完成后，可以使用docker images查看，多了一个rk399的镜像

```
zxc@zxc:~/zxc/docker$ docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
rk3399               latest              4b6336405b98       22 minutes ago     2.47GB
```

进入到docker环境，如果想退出后保留环境，请去掉--rm；使用--device挂载usb设备就可以在docker中访问了。

```
1 docker run -ti --rm --privileged --device /dev/bus/usb -v ./:/workspace,
```

2.编译烧录工具

在windows下官方提供了RKDevTool用于烧录，但是在Linux中编译，windows下烧录很麻烦，官方在Linux提供了另一种工具`rkdeveloptool`，但似乎只能自己去编译，并没有提供编译好的镜像。

进入到`rkdeveloptool`目录(注：这个已经在上面的环境中提供，就是那几个注释掉的git库，以下不再赘述)，依次执行：


```
1 ./autogen.sh
2 ./configure
3 make
```

运行`./rkdeveloptool`可以看见命令，其实这个编译坑主要在于环境，上面已经提供了。

3.构建官方loader

我在这里解释一个东西，因为我发现有一些文章里面弄错了这个。如果有使用官方的`RKDevTool`下载的，可以发现第一行是绿色的，这个并不烧录到emmc上，而是首先将其下载到ram上执行，执行这个后会初始化emmc，usb等硬件，然后接收数据烧录到emmc上。



所以说，那个地址改任何一个值都可行的，这个会被忽略；idbloader uboot必须是固定地址，因为这个是按地址加载，trust.img在主线中是不需要的。

现在来构建官方的工具，进入到`rkbin`，执行：

```
1 ./tools/boot_merger ./RKBOOT/RK3399MINIALL.ini
```

如果是其他的rk的处理器，请选择正确的.ini文件。

运行后获得：`rk3399\_loader\_v1.30.130.bin`，这个文件用在ram临时运行，烧录镜像到emmc。

在执行：

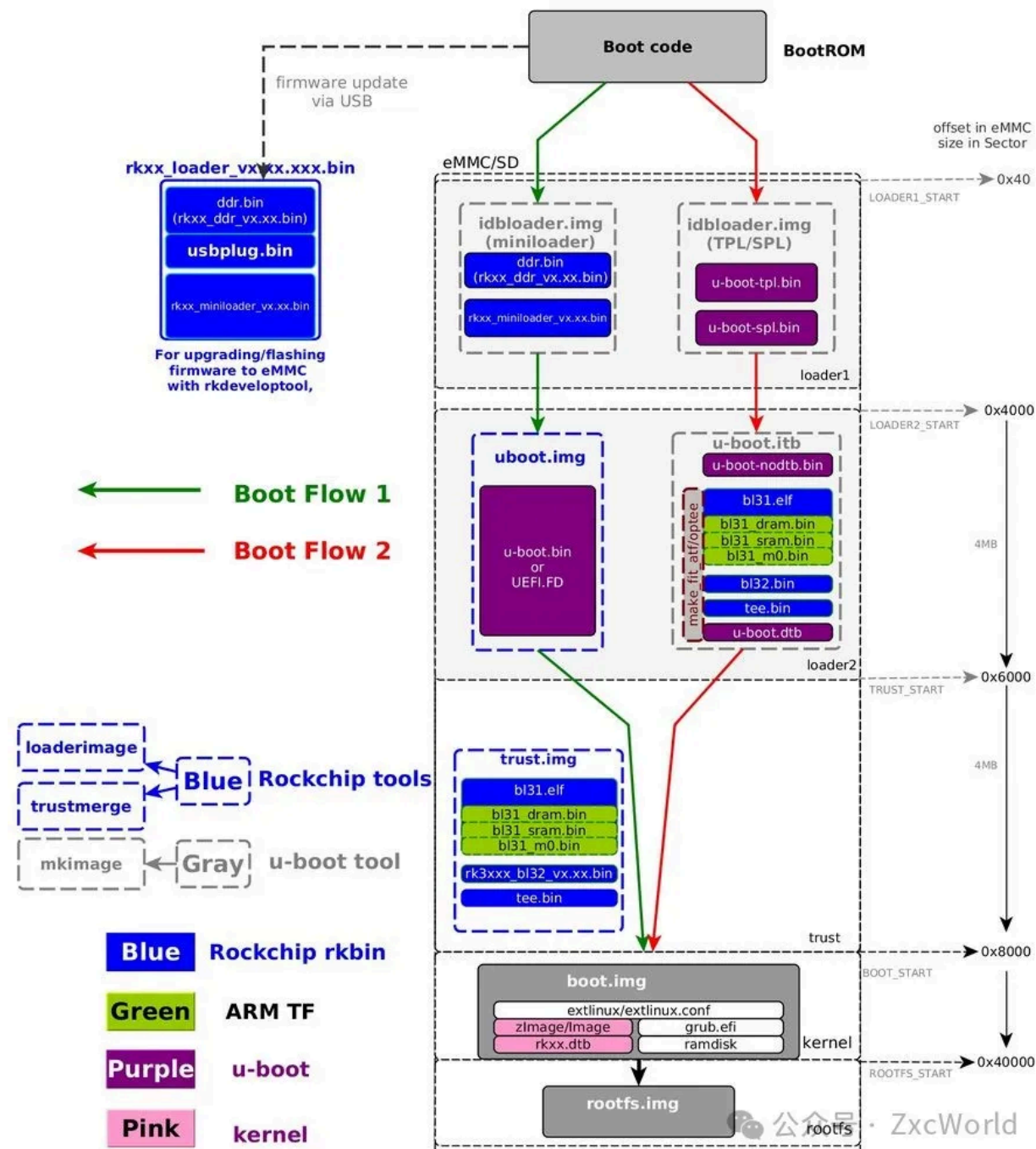
```
1 ./tools/trust_merger ./RKTRUST/RK3399TRUST.ini
```

运行后获得：`trust.img`，这文件是官方的启动方式中：atf和tee的镜像，官方会从这里加载atf和tee，主线uboot不需要。

4.构建uboot

4.1启动流程

先来个官方的流程图



如图片所示，启动其实分为3个阶段，BL1,BL2,BL3

BL1: 是最初的启动加载器，负责加载其他引导加载程序（如U-Boot）。

BL2: 对应idbloader,这一阶段在rk3399上，有两个分支，之所以有两个，是因为rk官方有一个闭源的路线(绿色的)，红色的是uboot的tpl。在这一阶段主要都是初始化设备，然后加载真正的uboot。

BL3：官方的路线中，bl3是uboot.img，这个具体的细节没有，但是可以猜测的是，会加载trust.img镜像中的东西运行。

在开源的uboot路线中，执行的uboot.itb，这个细节比较完整，这个阶段又分为了三个，BL31，BL32，BL33：

BL31：运行了ATF，ATF主要负责：负责处理 **PSCI 调用**（比如启动其他 CPU 核、挂起、重启等）；提供 **Secure Monitor Call（SMC）接口**，给 Linux 内核或 TEE 使用；同时ATF也会负责对uboot进行安全校验。

BL32：就是加载TEE了，被 ATF 启动，运行在 EL1 Secure world，提供加密服务、认证、安全存储等，这个TEE在那种有wifi的微处理器也是常见的，提供一个安全独立的运行环境。

BL33：就运行了uboot了。

这里就和之前的A33，或者后面将更新的v3s有较大区别，需要提供ATF和TEE。

4.2编译ATF

进入`trusted-firmware-a`目录：

```
1 make CROSS_COMPILE=aarch64-none-linux-gnu- PLAT=rk3399 bl31
```

编译好后，在目录build/rk3399/release/bl31/会生成bl31.elf：

可以将这个文件拷贝到uboot目录下，方便找到：

```
1 cp build/rk3399/release/bl31/bl31.elf ../u-boot/
```

4.3编译TEE

进入`optee\_os`目录，运行：

```
1 make -j1 CROSS_COMPILE64=aarch64-none-linux-gnu- CROSS_COMPILE32=arm-none-linux-gnueabi- zxc-rk3399_defconfig
```

编译完成后生成tee文件，同样将`tee.bin`拷贝到uboot下：

```
1 cp out/arm/core/tee.bin ../u-boot/
```

4.4编译uboot

设备先随便找一个，我这个板子我忘记将原始的镜像down下来了，又没有原理图，我无法确定设备了。使用orangepi就可以，主要是ddr参数要对，这个ddr参数一定要选择正确的那个，orangepi默认是不适配我这个板子的，启动uboot时会报错ram初始化有问题，我这个板子由于是咸鱼捡的，我就一个一个试了，也没几个；emmc节点要有，这个一般都有。其他的嘛，就不怎么重要，外设顶多初始化错误，不影响内核启动。哦对了，还有一个usb节点要改成设备模式，主要是使用uboot的ums功能，下面会介绍。

```
1 make CROSS_COMPILE=aarch64-none-linux-gnu- zxc-rk3399_defconfig
```

在这里开启`ums`,这个功能是将mmc变成块设备，我这里是为了方便后面上传内核和根文件。依次开启下面的)：

```
1 make -j1 CROSS_COMPILE64=aarch64-none-linux-gnu- menuconfig
```

```
1 CMD_USB_MASS_STORAGE
2 CONFIG_USB_FUNCTION_ROCKUSB
3 CONFIG_CMD_ROCKUSB
4 CONFIG_USB_GADGET
```

然后设备树这个ubs节点要改成设备模式，这样就能正确启用ums了：

```
1 &usbdrd_dwc3_0 {
2     status = "okay";
3     dr_mode = "peripheral";
4 };
```

编译：

```
1 make CROSS_COMPILE=aarch64-none-linux-gnu- BL31=./bl31.elf TEE=./tee.bin
```

编译完成后，获得`idbloader.img`,`u-boot.itb`。

4.5烧录uboot

将板子进入markdown模式。

先烧录loader到内存运行,每次烧录时，都必须先执行这个：

```
1 sudo ./rkdeveloptool db ../rkbin/rk3399_loader_v1.30.130.bin
```

再烧录idbloader.img到0x40，64即0x40：

```
1 sudo ./rkdeveloptool wl 0x40 ../u-boot/idbloader.img
```

再烧录u-boot.itb到0x4000：

```
1 sudo ./rkdeveloptool wl 0x4000 ../u-boot/u-boot.itb
```

重启：

```
1 sudo ./rkdeveloptool rd
```

这样uboot就启动了，可以使用串口控制台操作uboot了。

内核和debian根文件我再开一篇吧，似乎有点长了。

作者提示: 个人观点，仅供参考

修改于2025年5月18日